AI Forward Deployed Engineer — Five-Project Open-Source Program

> **Status: program of record.** This is the canonical 22-week, 440-hour AI FDE curriculum. The LocalStack edition remains an alternative under evaluation.

Executive summary

This candidate program prepares experienced software engineers to work as AI Forward Deployed Engineers: discovering ambiguous client problems, scoping delivery, building and evaluating production-shaped AI systems, managing stakeholders, and handing a bounded solution into operation. It is project-first and includes all just-in-time theory, implementation, documentation, review, and defense work within **22 weeks × 20 hours/week = 440 hours**.

Projects 1–4 and Phase 5.A run locally with Docker Compose and an open-source stack. They do not require a cloud account or expose a public endpoint. Phase 5.B first passes the same local acceptance path and then deploys one temporary, protected endpoint in a course-managed AWS sandbox. The total per-student program allowance remains **$20 for approved LLM APIs and $180 for AWS**.

This is not an entry-level or zero-to-hero program. Admission requires a practical assessment of Python, Git, REST APIs, SQL and relational databases, Docker, async service behavior, debugging an unfamiliar instrumented codebase, written stakeholder communication, and named cloud concepts. Customer-facing grading is asynchronous and deterministic, using role cards, fixed transcripts, checkpoint-triggered injects, evidence templates, recorded or written defenses, published acceptance floors, and common rubric anchors. An optional stakeholder chatbot may be used only for unassessed practice.

Required local environment

- **Hardware:** 16 GB RAM minimum using the assessed reduced profile; 32 GB recommended; 8 CPU cores and 80 GB free disk. Local GPU hardware is optional and never affects grading.
- **Operating system:** macOS, Linux, or Windows with WSL2.
- **Runtime profiles:** every project ships one pinned assessed Compose profile that must remain within a 10 GB container-memory budget. Optional observability or exploration profiles are started separately and are never required for grading.
- **Inference path:** a quantized model of no more than 3B parameters through Ollama is the assessed local path. The deterministic provider emulator is the grading fallback when local inference is unavailable. Local vLLM is optional exploration and cannot supply assessed latency or SLO evidence.
- **Admission preflight:** candidates run a supplied hardware, container, image-pull, and smoke-test check before enrollment. This is an admission requirement, not an uncounted program week.

Production-readiness dependencies

Before cohort authoring begins, the program must publish the scenario-pack contract, a versioned supplied-asset inventory, one `FIDELITY.md` per emulator or substitute service, five project rubrics with binary Must-Haves and Recommendations, pinned starter images, the Phase 5.A acceptance-gate schema, and the Phase 5.B verifier schema. These are program-build dependencies and do not add student hours.

Open-source delivery stack

- **Orchestration and application runtime:** Docker Compose, Python, FastAPI, PostgreSQL, Redis.
- **Object and document storage:** MinIO using an S3-compatible interface.
- **Search and retrieval:** PostgreSQL/pgvector and PostgreSQL full-text search with supplied configuration.
- **Messaging:** RabbitMQ with bounded retry and dead-letter patterns.
- **Identity and secrets:** preconfigured development-mode Keycloak and file-mounted Docker Compose secrets.
- **AI:** LangGraph, Ollama, manual Ragas evaluation, and Arize Phoenix.
- **Observability and security:** OpenTelemetry, Prometheus, Grafana, Tempo, Semgrep, Trivy, and Gitleaks.
- **Cloud transition:** supplied AWS templates and adapters in Phase 5.B; students are not asked to build a general cloud platform.

Program and project summary

Project	Weeks	Hours	Engagement outcome
1. Client Discovery and System Diagnostics	1–3	60	Diagnose an unreliable AI-enabled workflow and recommend a bounded next decision.
2. Enterprise Data Integration and Hybrid RAG	4–7	80	Deliver and defend a retrieval system over messy enterprise data.
3. Reliable AI Service Delivery	8–11	80	Stabilize an AI service under provider failures and changing requirements.
4. Security, Guardrails and Governance Approval	12–15	80	Secure a high-value AI workflow and obtain a defensible risk decision.
5. Bounded Agent Workflow and AWS Capstone	16–22	140	Build and accept one bounded, auditable agent workflow in Phase 5.A, then deploy, demonstrate, hand back, and tear down that release in Phase 5.B.
Total	**22**	**440**	**Five sequential field-delivery engagements.**

Program delivery contract

- Every task estimate includes just-in-time theory, implementation, documentation, review, and defense.
- Every project produces a client-facing decision or communication artifact.
- Projects 1–4 and Phase 5.A are locally runnable and graded; no public endpoint or cloud deployment is allowed.
- The program follows one continuous product lineage across five physical starter repositories. Project 1 starts from repository 1. Before each of Projects 2–5, TripleTen provides the next repository from the correct reference solution to the previous project, so an earlier student defect does not propagate. Phase 5.A produces the accepted capstone release tag in repository 5; Phase 5.B deploys, accepts, hands back, demonstrates, and tears down that exact tagged release.
- Published and held-out scenarios use the same interface but separate evidence. Reviewers grade decisions and evidence, not improvisational performance.
- Student-defined success criteria, SLOs, and acceptance conditions must meet or exceed a published project floor. Grades use the common floor plus the quality of the student's justification.
- The Project 5 rubric publishes separate Phase 5.A gate Must-Haves and Phase 5.B deployment, Final Demo Day, and teardown Must-Haves so gate acceptance, verifier verdicts, and program completion are not conflated.
- Exactly one assessed implementation is pinned per capability. Named alternatives are optional exploration and never affect grading or reviewer support.
- Projects 1–4 each end with a required **Instructor Presentation / Review**. Project 5's Instructor Presentation / Review occurs at the end of Phase 5.A and serves as the technical acceptance gate for Phase 5.B. The instructor applies the published project or phase Must-Haves and Recommendations and records feedback and the acceptance decision.
- Five graded Instructor Presentation / Review recordings are required: the Project 1 diagnostic handoff, Project 2 SoW defense, Project 3 operational handoff, Project 4 executive risk decision, and Phase 5.A technical acceptance gate. Each is one take of at most 10 minutes; the existing Project 2 and Project 4 defenses are their project reviews, not additional recordings. Production polish is not assessed.
- For Projects 1–4, a review may require resubmission of affected evidence within the published resubmission window; it does not reopen implementation work because the next project begins from TripleTen's reference checkpoint. A failed Phase 5.A Must-Have must be remediated and re-accepted before the Week 20 AWS provisioning deadline.
- During Phase 5.B, students may use instructor office hours for formative assistance with deployment evidence, acceptance interpretation, and Final Demo Day preparation. This assistance is not another graded presentation, does not change the published acceptance floor, and does not transfer ownership of student decisions or artifacts to the instructor.
- **Final Demo Day** is required for program completion and closes Project 5 after Phase 5.B. It occurs while the protected endpoint is available and before mandatory teardown. Each student delivers a 20-minute end-to-end demonstration followed by up to 10 minutes of questions covering business value, workflow behavior, architecture, authority and safety boundaries, evaluation evidence, AWS operations, cost, limitations, and the next recommendation. Production polish is not assessed.
- Optional live discovery and constraints-change drills may run during office hours, but attendance and performance do not affect completion or grading.
- Application code uses conformance-tested `ObjectStore`, `JobQueue`, `SecretProvider`, and `ModelProvider` ports. Vendor SDK clients remain inside supplied adapters; Phase 5.B changes adapters and configuration rather than business logic.
- Open-source services teach portable interfaces and operational reasoning. Students must state what local evidence does not prove about the corresponding AWS service.
- Projects 1–4 use no hosted LLM API. The program-issued, metered $20 allowance is split into $5 for a fixed Phase 5.A manual evaluation sample and $15 for Phase 5.B endpoint, held-out, and Final Demo Day traffic. Students never use personal API accounts or payment methods. Pre-launch cost validation must include the required Final Demo Day traffic profile.
- The program provisions no AWS GPU. Fine-tuning implementation is out of scope; students explain when it would be justified and why retrieval, prompting, or tool constraints are the correct levers for the assessed workflow.
- The five portfolio artifacts are: a traced client workflow and diagnosis, an undocumented legacy-integration specification and defensive client, unhappy-path resilience evidence, a deterministic evaluation suite with manual judged analysis, and a bounded agent with an audit trail.

Repository checkpoint model

Project	Physical repository	Starting checkpoint
1	Repository 1	Supplied initial legacy system
2	Repository 2	TripleTen reference solution through Project 1
3	Repository 3	TripleTen reference solution through Project 2
4	Repository 4	TripleTen reference solution through Project 3
5	Repository 5	TripleTen reference solution through Project 4; Phase 5.A produces the accepted release tag reused in Phase 5.B

At each reset, students receive the correct prior reference solution plus their own accepted client-facing artifacts and decision records. They compare the reference checkpoint with their prior implementation, but previous code defects cannot block the next project.

Project 1 — Client Discovery and System Diagnostics

Project summary

Students enter an unreliable AI-enabled enterprise workflow as an FDE, not as a greenfield developer. They clarify the business problem, navigate a supplied local system, diagnose a seeded failure, and defend an evidence-backed recommendation. The emphasis is discovery, evidence triage, ownership, and communication under ambiguity.

- **Schedule:** Weeks 1–3 / 60 hours
- **Local environment:** the pinned reduced Docker Compose profile with FastAPI, PostgreSQL, Redis, OpenTelemetry, Prometheus, Grafana, and Tempo.
- **Project evidence:** discovery record, repaired system, before/after evidence, ADR, recommendation playback, and diagnostic handoff.

Task 1.1 — Engagement kickoff and stakeholder discovery (14 hours)

- **[Summary]** Establish what the client is trying to achieve before investigating implementation details.
- **[Skills]** Customer discovery, active listening, problem framing, stakeholder mapping, assumption management, success criteria.
- **[Tools]** Asynchronous role cards, fixed transcript, checkpoint-triggered injects, discovery template, decision log.
- Trace users, workflow, desired business outcome, constraints, and known exceptions.
- Separate reported symptoms, assumptions, and facts requiring technical validation.
- Capture open questions, decision owners, competing priorities, and measurable success criteria.
- Produce a discovery record from a deterministic asynchronous simulation.

Task 1.2 — Runtime orientation and evidence triage (10 hours)

- **[Summary]** Understand the supplied platform quickly enough to investigate one important request path.
- **[Skills]** Codebase orientation, request tracing, log and metric interpretation, architecture reading, technical prioritization.
- **[Tools]** Pinned Docker Compose profile, FastAPI, PostgreSQL, Redis, OpenTelemetry, Prometheus, Grafana, Tempo.
- Run the supplied API, worker, database, cache, and observability services locally.
- Follow one request through code, logs, metrics, and a distributed trace.
- Use the supplied architecture map rather than producing a complete C4 model.
- Treat monolith-versus-microservices as a decision prompt, not a baseline theory module.

Task 1.3 — Diagnose and repair the seeded failure (14 hours)

- **[Summary]** Reproduce the client-visible failure and make one evidence-backed repair.
- **[Skills]** Reproduction, hypothesis testing, bottleneck analysis, observability repair, validation, causal reasoning.
- **[Tools]** Pytest, k6, OpenTelemetry, Prometheus, Grafana, Tempo, deterministic provider emulator.
- Run one bounded, deterministic load and provider-latency scenario.
- Repair one trace, metric, or dashboard defect selected by the scenario.
- Identify the first supported bottleneck and reject one plausible false explanation.
- Re-run the same scenario and record before-and-after evidence.

Task 1.4 — Problem framing and recommendation playback (12 hours)

- **[Summary]** Translate technical evidence into a decision the stakeholder can act on.
- **[Skills]** Technical synthesis, option framing, effort estimation, business communication, audience adaptation.
- **[Tools]** Recommendation template, lightweight estimation worksheet, architecture decision record.
- Rewrite the initial problem statement using verified workflow evidence.
- Present bounded alternatives, including a no-AI or process-change option when appropriate.
- Estimate effort and capacity using explicit assumptions.
- Record separate engineering-detail and business-impact explanations.

Task 1.5 — Diagnostic handoff (10 hours)

- **[Summary]** Leave the client with an auditable diagnosis and a bounded next decision.
- **[Skills]** Ownership, handoff writing, risk communication, recommendation defense, scope control.
- **[Tools]** Git, pull request, ADR, evidence packet, timed written defense.
- Submit the repair, evidence packet, concise ADR, and updated workflow trace.
- Record unresolved risks, excluded work, and next discovery questions.
- Respond in writing to a timed stakeholder challenge without inventing evidence.
- Defend ownership of the recommendation through an evidence-linked decision record.
- **[Instructor Presentation / Review]** Present the final evidence chain, repair, ADR, unresolved risks, and ownership decision without repeating the earlier recommendation playback. The instructor records feedback and either accepts the evidence or requires evidence resubmission within the published window.

Project 2 — Enterprise Data Integration and Hybrid RAG

Project summary

Students scope and deliver a retrieval system over messy client documents without turning the engagement into a platform rewrite. They work through data ownership and access constraints, construct a reproducible ingestion path, implement hybrid retrieval, evaluate failure modes, and defend a statement of work.

- **Schedule:** Weeks 4–7 / 80 hours
- **Local environment:** Docker Compose, MinIO, PostgreSQL/pgvector, PostgreSQL full-text search, supplied document parser, local embeddings, and a supplied cross-encoder.
- **Project evidence:** data discovery record, ingestion report, retrieval API, golden set, evaluation report, and defended SoW.

Task 2.1 — Data and use-case discovery (12 hours)

- **[Summary]** Determine whose decision retrieval supports and which data can be used safely.
- **[Skills]** Data discovery, domain interviewing, ownership mapping, access analysis, acceptance-criteria design.
- **[Tools]** Stakeholder role cards, data inventory, access matrix, golden-query template.
- Analyze asynchronous responses from domain, data-owner, and compliance stakeholders.
- Document source ownership, access constraints, freshness needs, and failure consequences.
- Define measurable retrieval acceptance criteria before choosing technical methods.
- Identify one process or data-quality problem that RAG should not conceal.

Task 2.2 — Messy-document ingestion (20 hours)

- **[Summary]** Build a reproducible ingestion pipeline for a deliberately bounded enterprise document set.
- **[Skills]** Ingestion design, metadata normalization, deduplication, chunking, embedding, data-quality validation.
- **[Tools]** MinIO, supplied PDF/HTML parser, PostgreSQL, Python, local embedding model, Docker Compose.
- Use the supplied robust parser with text-based PDFs and clean, bounded table structures; OCR library selection is out of scope.
- Store source documents in MinIO and normalize metadata, duplicate identifiers, malformed content, and access labels.
- Implement bounded chunking and local embedding through the supplied pipeline scaffold.
- Produce ingestion-quality evidence for published and held-out fixtures.
- Cite the `ObjectStore` fidelity note and state which S3 behaviors the local result does not prove.

Task 2.3 — Hybrid retrieval service (24 hours)

- **[Summary]** Implement the smallest production-shaped retrieval boundary that meets the agreed use case.
- **[Skills]** Dense and sparse retrieval, reranking, authorization metadata propagation, API extension, migration use.
- **[Tools]** PostgreSQL/pgvector, PostgreSQL full-text search, supplied reciprocal-rank fusion and cross-encoder wrappers, FastAPI, Alembic.
- Combine pgvector dense retrieval with PostgreSQL full-text search through the supplied fusion function.
- Rerank hybrid candidates locally before returning the final result set.
- Preserve source metadata and authorization attributes through retrieval.
- Extend the supplied versioned API and reuse provided migrations, repositories, and connection pooling.
- Choose and justify candidate depth, fusion parameters, and the rerank cut-off; show their measured latency and quality effects.

Task 2.4 — Retrieval evaluation and failure analysis (14 hours)

- **[Summary]** Demonstrate when retrieval succeeds, when it fails, and why.
- **[Skills]** Golden-set design, retrieval metrics, latency analysis, error attribution, evidence-based iteration.
- **[Tools]** Pytest, evaluation fixtures, recall/precision metrics, OpenTelemetry, supplied reporting script.
- Build a compact golden query set with relevance expectations.
- Measure retrieval quality and latency against agreed criteria.
- Attribute misses to parsing, chunking, metadata, dense retrieval, keyword retrieval, or reranking.
- Recommend one bounded improvement supported by evidence.
- Meet or exceed the published retrieval-quality and latency floor on held-out fixtures.

Task 2.5 — Scope and statement-of-work defense (10 hours)

- **[Summary]** Convert discovery and technical findings into a delivery agreement.
- **[Skills]** SoW writing, scope control, estimation, negotiation, change rejection, acceptance definition.
- **[Tools]** SoW template, estimation worksheet, assumption log, timed stakeholder inject, recorded defense.
- Define outcomes, deliverables, assumptions, exclusions, acceptance criteria, and responsibilities.
- Estimate the next increment and identify its largest uncertainty.
- Process a stakeholder request that expands scope without moving time or budget.
- **[Instructor Presentation / Review]** Record the graded SoW defense as the Project 2 review, covering outcomes, exclusions, assumptions, estimate, acceptance criteria, what will not be built, and the scope-change decision. The instructor records feedback and either accepts the evidence or requires evidence resubmission within the published window; no additional presentation is produced.

Project 3 — Reliable AI Service Delivery

Project summary

Students stabilize a retrieval-enabled AI service whose provider behavior and client priorities change under load. They operate local inference and deterministic provider paths, implement a bounded resilience pattern, connect it to an SLO, renegotiate a changed requirement, and hand over an executable incident process.

- **Schedule:** Weeks 8–11 / 80 hours
- **Local environment:** Docker Compose, Ollama, deterministic provider and undocumented legacy-service emulators, RabbitMQ, OpenTelemetry, Prometheus, and Grafana.
- **Project evidence:** resilient service, scenario results, SLO and alert, failure timeline, approved change record, incident communications, and runbook.

Task 3.1 — Local AI runtime and service boundary (18 hours)

- **[Summary]** Operate the platform through portable provider boundaries and derive a defensive contract for one undocumented legacy service.
- **[Skills]** Provider abstraction, legacy integration, schema tolerance, permission and rate-limit discovery, fallback reasoning.
- **[Tools]** Docker Compose, Ollama, FastAPI, deterministic provider emulator, supplied undocumented legacy-service emulator.
- Run the supplied multi-service environment locally.
- Use one provider adapter across local inference and deterministic scenarios.
- Discover an inconsistent response schema, hidden rate limit, stale field, and denied operations through observed behavior.
- Produce a derived interface specification and a defensive client with schema tolerance and bounded backoff.
- Explain Kubernetes, managed inference, and fine-tuning only as architecture alternatives relevant to the service.

Task 3.2 — Provider resilience (20 hours)

- **[Summary]** Keep the client workflow safe during latency, rate-limit, malformed-response, and outage scenarios.
- **[Skills]** Deadlines, retry and backoff, circuit breaking, fallback, idempotency, asynchronous failure handling.
- **[Tools]** RabbitMQ, provider emulator, supplied resilience library, Pytest, OpenTelemetry.
- Configure the supplied circuit-breaker-with-fallback pattern and use bounded retry only for scenario-approved transient failures.
- Enforce deadlines, structured failures, and idempotent behavior where required.
- Implement one RabbitMQ dead-letter path and submit evidence of failed-message recovery.
- Prove behavior through deterministic provider scenarios.
- Cite the `JobQueue` fidelity note and distinguish RabbitMQ delivery semantics from the supplied AWS adapter.

Task 3.3 — SLO and failure lab (18 hours)

- **[Summary]** Connect one operational objective to a client-visible failure and recovery workflow.
- **[Skills]** SLO design, alerting, reversible failure testing, incident timelines, residual-risk analysis.
- **[Tools]** Prometheus, Grafana, OpenTelemetry, supplied failure injector, k6.
- Define one user-impact SLO and one actionable alert.
- Run one reversible failure lab and capture detection, mitigation, and recovery evidence.
- Validate the selected resilience behavior under failure.
- Document remaining failure modes without constructing a broad chaos suite.
- Meet or exceed the published availability and recovery floor; justify any stricter student-defined target.

Task 3.4 — Changing-requirements simulation (12 hours)

- **[Summary]** Re-plan delivery after a stakeholder changes a reliability requirement.
- **[Skills]** Clarification, impact analysis, negotiation, prioritization, change control, decision documentation.
- **[Tools]** Timed asynchronous inject, impact template, change log, timed written response.
- Clarify the outcome, urgency, decision owner, and hidden trade-off.
- Assess effects on scope, risk, schedule, and operational cost.
- Negotiate which commitment moves out when the new requirement moves in.
- Record the approved change rather than silently absorbing it.
- During the same task, triage one Project 2 client escalation against the active Project 3 commitment and record the context-switching trade-off.

Task 3.5 — Incident communication and operational handoff (12 hours)

- **[Summary]** Communicate a failure honestly and leave an executable recovery process.
- **[Skills]** Incident communication, audience adaptation, uncertainty communication, runbook design, escalation planning.
- **[Tools]** Incident templates, runbook template, timeline evidence, timed written briefing.
- Write separate engineering and stakeholder incident updates.
- Write a short stakeholder briefing while recovery information remains uncertain.
- Deliver a focused detection, mitigation, rollback, and escalation runbook.
- Convert the failure lesson into one reusable delivery pattern.
- **[Instructor Presentation / Review]** Present the SLO evidence, failure response, stakeholder communication, runbook, and remaining risks. The instructor records feedback and either accepts the evidence or requires evidence resubmission within the published window.

Project 4 — Security, Guardrails and Governance Approval

Project summary

Students secure one high-value AI workflow and make the result inspectable by engineering, security, compliance, and business stakeholders. They select proportionate controls, implement one bounded security boundary, test it against held-out attacks, and obtain a documented risk decision without overstating educational evidence.

- **Schedule:** Weeks 12–15 / 80 hours
- **Local environment:** preconfigured development-mode Keycloak, file-mounted Docker Compose secrets, Semgrep, Trivy, Gitleaks, OpenTelemetry, and supplied attack scenarios.
- **Project evidence:** stakeholder risk record, threat model, tested controls, control matrix, residual-risk register, and executive decision.
- **Evidence boundary:** the project demonstrates security-control design and integration against development services. It does not demonstrate production hardening, key custody, an audit, certification, or legal compliance. Every control-matrix row states what its evidence does and does not prove.

Task 4.1 — Security and governance discovery (12 hours)

- **[Summary]** Identify actors, sensitive data, decision rights, and risk tolerance.
- **[Skills]** Security discovery, workshop facilitation, conflict reconciliation, risk treatment, governance ownership.
- **[Tools]** Stakeholder role cards, data-flow template, risk register, asynchronous workshop injects.
- Reconcile engineering, security, compliance, and business expectations.
- Map sensitive data, availability needs, audit expectations, and decision rights.
- Distinguish risks requiring control, acceptance, transfer, or feature removal.
- Record conflicts and accountable decision owners.

Task 4.2 — Threat model and control selection (18 hours)

- **[Summary]** Model one workflow deeply enough to choose proportionate controls.
- **[Skills]** STRIDE, trust-boundary analysis, abuse-case design, prompt-injection analysis, control selection.
- **[Tools]** Threat-model template, data-flow diagram, OWASP LLM guidance, control decision record.
- Identify assets, actors, trust boundaries, and high-impact abuse cases.
- Emphasize prompt injection and unsafe output handling.
- Select controls using evidence and reject unnecessary complexity.
- Treat legal frameworks as engineering-control mapping, not certification.

Task 4.3 — Implement the security boundary (30 hours)

- **[Summary]** Implement and verify the smallest control set needed for the workflow.
- **[Skills]** OIDC/RBAC integration, prompt and output controls, PII redaction, audit logging, secrets management, CI security.
- **[Tools]** Preconfigured Keycloak realm, Docker Compose secrets, FastAPI, supplied redactor, supplied pinned CI security workflow, Semgrep, Trivy, Gitleaks, Pytest.
- Protect one endpoint using the supplied development OIDC/RBAC scaffold.
- Add prompt-injection handling, output validation, PII redaction, and audit logging at one AI boundary.
- Move one hardcoded secret to a file-mounted Compose secret and configure the supplied deterministic CI security gate.
- Test published and held-out attack and data-leakage scenarios; explain remaining gaps.
- State what the development secret-store evidence does and does not prove about AWS secret custody and access enforcement.

Task 4.4 — Governance evidence and adversarial review (12 hours)

- **[Summary]** Make security claims inspectable by people who did not build the system.
- **[Skills]** Evidence mapping, adversarial review, AI-output verification, residual-risk writing, human-oversight design.
- **[Tools]** Control matrix, test evidence, risk register, supplied flawed AI-generated artifact.
- Link risks, controls, tests, evidence, and owners.
- Review a flawed AI-generated threat or compliance artifact for overclaims and omissions.
- Record residual risks and limits of the evidence.
- Define required human oversight and review triggers.

Task 4.5 — Executive risk decision (8 hours)

- **[Summary]** Obtain a go, conditional-go, or no-go decision without hiding uncertainty.
- **[Skills]** Executive communication, risk translation, boundary defense, decision facilitation, accountability.
- **[Tools]** Executive brief, recorded defense, timed pressure inject, decision record.
- Present value, material risks, implemented controls, and remaining exposure.
- Respond to pressure to describe educational evidence as production certification.
- Record the outcome, owner, conditions, and review date.
- **[Instructor Presentation / Review]** Record the graded executive risk defense as the Project 4 review, covering value, threat evidence, controls, residual risks, evidence boundaries, and the go, conditional-go, or no-go recommendation. The instructor records feedback and either accepts the evidence or requires evidence resubmission within the published window; no additional presentation is produced.

Project 5 — Bounded Agent Workflow and AWS Capstone

Project summary

Students complete one continuous capstone engagement in Repository 5. In Phase 5.A, they turn a vague automation request into one bounded, auditable LangGraph workflow and complete the project's Instructor Presentation / Review and local technical acceptance gate. In Phase 5.B, they deploy that exact accepted release through the supplied protected AWS path, receive formative instructor assistance during preparation, demonstrate acceptance evidence, prepare users and operators, complete Final Demo Day, and prove teardown.

- **Schedule:** Weeks 16–22 / 140 hours
- **Repository:** Repository 5, beginning from TripleTen's correct reference checkpoint through Project 4 and continuing across both capstone phases.
- **Project evidence:** workflow map, value hypothesis, bounded agent, audit trail, evaluation report, accepted release tag, protected endpoint, verifier report, acceptance decision, handback pack, Instructor Presentation / Review evidence, Final Demo Day evidence, final cost, and teardown proof.
- **Evidence boundary:** Phase 5.A local evidence establishes workflow behavior and deployment readiness but does not prove real-AWS behavior. Phase 5.B evidence applies only to the fixed course-managed profile and 14-day window; it does not prove production scale, durability, or unrestricted cloud operations.

Phase 5.A — Bounded Agent Workflow Under Ambiguity

Students define the authority boundary, implement the smallest useful agent flow, evaluate it with deterministic scenarios, process a late change, and defend the locally accepted release to technical and business audiences.

- **Phase schedule:** Weeks 16–19 / 80 hours
- **Local environment:** LangGraph, Project 2 retrieval, Ollama, Ragas, Arize Phoenix, PostgreSQL, and Docker Compose.
- **Phase evidence:** workflow map, value hypothesis, bounded agent, audit trail, evaluation report, approved change, Instructor Presentation / Review recording, technical readiness memo, instructor acceptance decision, and accepted release manifest.

Task 5.A.1 — Ambiguous client brief and workflow discovery (12 hours)

- **[Summary]** Turn a vague automation request into an observable workflow and decision problem.
- **[Skills]** Workflow discovery, ambiguity management, exception analysis, authority design, assumption tracking.
- **[Tools]** Process map, stakeholder role cards, timed injects, assumption log, decision inventory.
- Trace current handoffs, exceptions, failure costs, and human decision points.
- Identify where deterministic software is safer or cheaper than an agent.
- Define authority, escalation conditions, prohibited actions, and decision owners.
- Update assumptions as stakeholder information changes.

Task 5.A.2 — Scope and value hypothesis (10 hours)

- **[Summary]** Select a narrow agent use case with an evidence-based business outcome.
- **[Skills]** Value framing, scope reduction, cost estimation, acceptance design, rollback planning, autonomy negotiation.
- **[Tools]** Value hypothesis canvas, cost model, SoW addendum, acceptance checklist.
- Define value through revenue, risk, time, or operating cost.
- Estimate delivery and operating cost from named assumptions.
- Set acceptance, exclusion, rollback, and human-override conditions.
- Defend a smaller scope when broad autonomy is requested.
- Meet or exceed the published authority, override, and rollback floor.

Task 5.A.3 — Bounded agent implementation (24 hours)

- **[Summary]** Implement one auditable workflow that cannot exceed its agreed authority.
- **[Skills]** State-graph design, tool integration, schema validation, sandboxing, budget controls, human-in-the-loop design.
- **[Tools]** LangGraph, Pydantic, Project 2 retrieval API, Ollama, PostgreSQL, Docker Compose.
- Build one bounded multi-step tool flow rather than a general agent platform.
- Reuse Project 2 retrieval and add step, token, request, and tool limits.
- Preserve tool calls, state transitions, failures, and human decisions in an audit trail.
- Fine-tuning implementation is out of scope. In the technical defense, explain when fine-tuning would be justified and why prompting, retrieval, or tool constraints are the appropriate levers for this workflow.

Task 5.A.4 — Agent evaluation and observability (18 hours)

- **[Summary]** Prove useful behavior and expose important failures without relying on demo impressions.
- **[Skills]** Scenario-set design, agent evaluation, trace analysis, cost analysis, judge calibration, failure interpretation.
- **[Tools]** Supplied deterministic metric script, Ragas, Arize Phoenix, OpenTelemetry, Pytest, cached fixtures, bounded LLM-as-judge worksheet.
- Cover success, refusal, escalation, and recovery in a compact golden scenario set.
- Run bit-reproducible CI checks for schema validity, tool-sequence conformance, recall, refusal, escalation, and recovery using cached fixtures.
- Use Ragas and an LLM-as-judge manually on one fixed 20-case sample; report variance across three runs. Neither gates CI.
- Use Phoenix for trace inspection, latency, cost, tool behavior, and user-visible failure.

Task 5.A.5 — Instructor Presentation / Review and technical acceptance gate (16 hours)

- **[Summary]** Revise the solution under pressure and pass the instructor-reviewed technical acceptance gate that authorizes the deployment phase.
- **[Skills]** Change control, impact analysis, technical defense, evaluation-gap disclosure, deployment-readiness reasoning, evidence-based communication.
- **[Tools]** Change request, architecture evidence, evaluation report, Project 5 Phase 5.A Must-Haves and Recommendations, acceptance-gate schema, recorded Instructor Presentation / Review, release manifest.
- Process a late stakeholder request through explicit analysis and approval.
- Explain architecture, safety boundaries, evaluation gaps, and rejected alternatives to engineers.
- **[Instructor Presentation / Review]** Record one graded technical presentation covering the bounded workflow, authority limits, evaluation evidence, unresolved gaps, and deployment readiness. The instructor applies the published Phase 5.A gate and records an accepted or not-accepted decision.
- Submit a short readiness memo covering evaluation gaps, deferred work, deployment risks, and required operational controls; the final business and investment narrative belongs to Final Demo Day.
- Record approved change, deferred work, and uncertainty.
- If the gate is not accepted, remediate failed Must-Haves and obtain instructor re-acceptance before the Week 20 AWS provisioning deadline.
- On acceptance, create an immutable release tag and manifest recording its commit SHA and image digest. Phase 5.B may not provision AWS resources until the instructor decision and manifest are recorded.

Phase 5.B — AWS Deployment, Adoption and Handback

Students take the instructor-accepted Phase 5.A release through a supplied, protected AWS deployment path. They agree on responsibilities and cost controls, demonstrate acceptance evidence, prepare users and operators, use formative instructor support during preparation, complete Final Demo Day, and prove teardown. The phase teaches bounded cloud delivery—not general AWS administration.

- **Phase schedule:** Weeks 20–22 / 60 hours
- **Fixed cloud profile:** one course-managed AWS Organizations member account and one `t3.large` CPU host run Caddy, the application, worker, PostgreSQL/pgvector with full-text search, and Redis. S3 supplies object storage, SQS/DLQ supplies asynchronous delivery, and SSM Parameter Store supplies configuration. Keycloak, Phoenix, Prometheus, Grafana, LocalStack, MinIO, and RabbitMQ are not deployed.
- **Prohibited resources:** no GPU, NAT gateway, load balancer, managed database, managed search domain, second public IPv4 address, second always-on instance, student-created IAM principal, or untagged resource.
- **Cloud boundary:** one temporary protected endpoint and 14 consecutive days of reviewer access beginning at the first verified endpoint, which must occur by the end of Week 20. Reviewer turnaround is three business days. Final Demo Day occurs after any instructor-approved remediation deployment and no later than day 12 of the reviewer window; verified teardown completes by day 14 and inside the 22-week program.
- **Budget:** $180 maximum for AWS. The normal target is at most $80; remaining headroom covers exactly one instructor-approved remediation deployment and operational variance. Current prices and the profile must be validated in a course sandbox before launch.
- **Access and cleanup:** the program supplies constrained launch templates, a constrained student role, budget actions, the DNS hostname, Caddy TLS configuration, Session Manager access, automated teardown, and the verifier. Students never use personal AWS accounts or payment methods.
- **Phase evidence:** deployment agreement, protected endpoint, release-provenance verification, verifier verdict, handback pack, instructor feedback from formative preparation, Final Demo Day evidence, final cost, and teardown proof.

Task 5.B.1 — Capstone deployment agreement (8 hours)

- **[Summary]** Agree on responsibilities, acceptance evidence, cost limits, and exit conditions before using AWS.
- **[Skills]** Deployment planning, RACI definition, readiness assessment, cost governance, acceptance design, escalation planning.
- **[Tools]** Readiness checklist, RACI, supplied budget report, delivery plan, review SLA, teardown contract.
- Review security, support, data, access, dependencies, and decision owners.
- Confirm the $15 Phase 5.B API allocation, $180 AWS cap, 14-day reviewer window, three-business-day review SLA, approved region, tags, and teardown deadline.
- Define milestones, responsibilities, escalation paths, acceptance criteria, and rollback/no-go threshold.
- Confirm the instructor's recorded Phase 5.A acceptance decision and accepted release manifest before provisioning cloud resources.
- Confirm that any failed Project 4 Must-Have has been remediated and re-verified before deployment.

Task 5.B.2 — Protected AWS deployment (18 hours)

- **[Summary]** Deploy the tagged, locally accepted release through the fixed course-managed AWS profile.
- **[Skills]** Local-to-cloud translation, migration-gap analysis, protected deployment, secrets handling, least privilege, cost-aware operation, release verification.
- **[Tools]** Supplied launch templates and deploy script, AWS CLI, ECR, EC2, S3, SQS/DLQ, SSM Parameter Store, CloudWatch, Caddy.
- Run the supplied deploy script and launch templates; students do not author general-purpose infrastructure.
- Produce the first verified endpoint within the first 20 Phase 5.B hours; remaining deployment-task time is reserved for control verification and stabilization.
- Confirm through the supplied verifier that the deployed image digest matches the accepted Phase 5.A release manifest.
- Replace local adapters with supplied AWS adapters without changing business logic.
- Produce a migration-delta record naming each local/AWS mapping, configuration change, AWS-only policy or network requirement, and one failure mode that local evidence could not prove.
- Configure the program-issued hostname and trusted TLS, reviewer token, 30 requests/minute/token, 128 KiB request bodies, HTTPS-only ingress, no public SSH, IMDSv2, controlled egress, tags, synthetic data, and bounded retention.
- Demonstrate and handle at least one real-AWS policy-denial, throttling, consistency, or network-reachability failure.

Task 5.B.3 — Acceptance, operations and cost evidence (12 hours)

- **[Summary]** Demonstrate that the deployed system meets technical and business acceptance conditions.
- **[Skills]** Acceptance testing, cloud observability, failure triage, cost analysis, remediation prioritization, stakeholder communication.
- **[Tools]** Course verifier, CloudWatch, supplied budget report, held-out scenario pack.
- Run held-out functional, security, resilience, and teardown-readiness checks.
- Show quality, latency, failure, and cost evidence against agreed criteria.
- Triage one simulated acceptance failure and communicate its delivery impact.
- Keep remediation inside the endpoint, assessment window, and budget.
- Apply the published evidence thresholds to derive the verifier verdict—accept, conditional-accept, or reject; one instructor-approved remediation deployment is permitted before automated teardown.

Task 5.B.4 — Adoption and operational handback (8 hours)

- **[Summary]** Prepare the client organization to operate, govern, and appropriately use the system.
- **[Skills]** Adoption planning, enablement, runbook design, ownership transfer, limitation communication, success measurement.
- **[Tools]** Operator runbook, user guide, escalation map, limitations register, timed written enablement scenario.
- Identify user groups, workflow changes, adoption risks, and responsible owners.
- Produce operator guidance, user guidance, escalation paths, and known limitations.
- Respond to skeptical-user questions through a deterministic timed written simulation.
- Define post-launch measures for usage, value, quality, and safety.

Task 5.B.5 — Final Demo Day preparation and instructor support (6 hours)

- **[Summary]** Prepare one evidence-grounded Final Demo Day narrative and rehearse the protected-endpoint demonstration without creating another graded defense.
- **[Skills]** Demonstration design, technical-to-business synthesis, evidence discipline, constraint response, rehearsal, feedback incorporation.
- **[Tools]** Final Demo Day run-sheet template, architecture evidence, value brief, verifier verdict, cost report, protected endpoint, instructor office hours.
- Assemble the accepted technical, operational, cost, adoption, and limitation evidence into one end-to-end demonstration narrative.
- Prepare the protected-endpoint run sheet, audience transitions, question ownership, and a deterministic evidence replay for a program-side endpoint outage.
- Use optional formative instructor assistance to clarify evidence, rehearse the demonstration, and identify unsupported claims. The instructor does not produce or edit the student's decisions or artifacts.
- Incorporate feedback, finalize the 20-minute demonstration and question preparation, and preserve the approved scope and evidence boundaries.

Task 5.B.6 — Final Demo Day, teardown, portfolio and reusable field playbook (8 hours)

- **[Summary]** Demonstrate the completed capstone, close the engagement safely, and turn customer-specific learning into reusable delivery knowledge.
- **[Skills]** End-to-end demonstration, evidence-based communication, teardown verification, engagement closure, evidence retention, knowledge reuse, sensitive-data separation.
- **[Tools]** Protected endpoint, Final Demo Day evidence template, AWS CLI, supplied teardown script, cost report, resource inventory, field-playbook template.
- Complete the required **Final Demo Day** after any approved remediation and no later than day 12 of the reviewer window. Deliver the 20-minute end-to-end demonstration and respond to up to 10 minutes of questions about business value, workflow behavior, architecture, authority and safety boundaries, evaluation evidence, AWS operations, cost, limitations, and the next recommendation.
- Record the demonstration evidence required for program completion, then begin teardown.
- Remove and verify instances, volumes, public IPv4 addresses, tokens, parameters, traces, images, buckets, and endpoint records by day 14 of the reviewer window.
- Deliver final cost, teardown, acceptance, and unresolved-risk evidence.
- Separate reusable patterns from client-specific and sensitive information.
- Publish a sanitized capstone repository, reference-architecture write-up, discovery-to-handback playbook, and product-feedback record using synthetic data and excluding role-card content.
- Add the five auditable program artifacts and sanitized Final Demo Day evidence to a public portfolio index.

Workload verification

Project	Calculation	Hours
Project 1	3 weeks × 20	60
Project 2	4 weeks × 20	80
Project 3	4 weeks × 20	80
Project 4	4 weeks × 20	80
Project 5	7 weeks × 20	140
↳ Phase 5.A	4 weeks × 20	80
↳ Phase 5.B	3 weeks × 20	60
Program	**22 weeks × 20**	**440**